ADC and Analog Interfacing

Introduction to ARM Cortex-M Microprocessors — Chapter 11

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Matters
2. Sampling, Quantization, and Resolution
3. The STM32 ADC
4. Experiment: Reading a Potentiometer
5. Interrupt-Driven Conversion
6. From Codes to Engineering Units
7. Summary

Why This Chapter Matters

Temperature, light, sound and force are continuous. A processor works only with numbers. The ADC crosses that gap. Using it well is an engineering problem: how fast to sample, how much resolution you need, and how much of the answer you can trust.

After this chapter, you can:

1. Explain sampling, quantization and resolution; apply Nyquist.
2. Calculate the step size and convert codes to volts.
3. Configure a pin for analog input and set up the ADC.
4. Read by polling and by interrupt.
5. Choose the sampling time for a source impedance.
6. Scale a raw reading into engineering units.

© Munim Zabidi

By the end of this chapter you will know exactly why this code turns a voltage into a trustworthy number:

```
raw     = ADC1->DR;                    // 0 .. 4095
voltage = ((float)raw * 3.3f) / 4095.0f;
```

Questions this code raises, and where this chapter answers them:

Question raised	Answered in
What is a code of 2048?	Sampling and resolution
What does the ADC need before it converts?	The STM32 ADC
Why can the reading be wrong yet steady?	Resolution is not accuracy
How fast can it sample?	Interrupt-driven conversion

© Muním Zabidi

BEAMER EXTRA

Table of Contents

1 Why This Matters

2 **Sampling, Quantization, and Resolution**

3 The STM32 ADC

4 Experiment: Reading a Potentiometer

5 Interrupt-Driven Conversion

6 From Codes to Engineering Units

7 Summary

Two Approximations at Once

Sampling — discrete in time

You measure at instants, not continuously. Between samples, the signal is unknown.

Nyquist: sample at more than twice the highest frequency present, or the reconstruction is wrong.

Quantization — discrete in value

Each sample becomes one of a finite set of numbers. 12 bits gives $2^{12} = 4096$ levels, 0 to 4095.

The error is up to half a step — inherent, not a defect.

Aliasing is not recoverable

Sample a 600 Hz tone at 1 kHz and it reappears as 400 Hz — and no amount of later processing can tell it from a real 400 Hz signal. The fix is an anti-alias filter in hardware, before the ADC.

From Count to Voltage

$$V_{\text{in}} = \frac{\text{count}}{4095} \times V_{\text{ref}}$$

Count	Voltage ($V_{\text{ref}} = 3.3\,\text{V}$)	Meaning
`0`	0.00 V	Input at ground
`1`	0.81 mV	One step — the smallest detectable change
`2048`	1.65 V	Half scale
`4095`	3.30 V	Full scale

Resolution is a voltage, not a bit count

"12-bit" sounds abstract. What it *means* is that the smallest change you can see is $3.3/4095 \approx 0.8\,\text{mV}$. Anything finer is invisible to your program.

© Muním Zabidi

A potentiometer sits at mid-travel. The 12-bit ADC reads `raw = 2048`, with $V_{REF} = 3.3\,\text{V}$.

Your prediction

- What voltage does that code represent?
- What is one LSB, in millivolts?
- If the real pin voltage is 1.60 V, what could explain the gap?

© Munim Zabidi

BEAMER EXTRA

A potentiometer sits at mid-travel. The 12-bit ADC reads `raw = 2048`, with $V_{REF} = 3.3\,V$.

Your prediction

- What voltage does that code represent?
- What is one LSB, in millivolts?
- If the real pin voltage is 1.60 V, what could explain the gap?

The answer

$V = 2048 \times 3.3/4095 = 1.650\,V$. One LSB $= 3.3/4095 \approx 0.806\,mV$. A 50 mV gap is 62 LSB: look at the reference voltage, the sampling time and the circuit, not at the arithmetic.

Table of Contents

1 Why This Matters

2 Sampling, Quantization, and Resolution

3 **The STM32 ADC**

4 Experiment: Reading a Potentiometer

5 Interrupt-Driven Conversion

6 From Codes to Engineering Units

7 Summary

The STM32F446RE has three independent 12-bit successive-approximation (SAR) ADCs. Each can sample up to 16 external channels, and several internal sources: the temperature sensor, the battery monitor and a precision reference.

Mode	What it does
Single conversion	One conversion, then it stops.
Continuous	A new conversion starts when the last one ends.
Scan	Converts a programmed sequence of channels in order.
Alignment	Places the 12-bit result in a 16-bit register: right-aligned (default) or left-aligned.

© Muním Zabidi

ADC and GPIO Registers Used in This Chapter

Register	Bit	Name	Function
GPIOx_MODER	pin mode		Analog mode (11). Disconnects the Schmitt trigger and pull resistors.
ADC_CR1	25:24		Resolution: 00=12, 01=10, 10=8, 11=6 bit
	8		Scan mode
	5	EOCIE	End-of-conversion interrupt enable
ADC_CR2	0	ADON	Powers the ADC
	1	CONT	Continuous conversion
	11	ALIGN	Result alignment
	30	SWSTART	Triggers a conversion
ADC_SMPR1/2	3 bits/ch		Sampling time, 000 (3 cycles) to 111 (480). Channels 0–9 use SMPR2.
ADC_SQR1/2/3	23:20, 4:0	L, SQ1	Sequence length; first channel
ADC_SR	1	EOC	Set when a result is ready
ADC_DR	15:0		The result. Reading it clears EOC.

- The ADC charges an internal capacitor from your signal before converting.
- Sampling time is how long it is given to charge — 3 to 480 cycles.

```
ADC1->SMPR2 |= (7U « 0); // 480 cycles
```

Too short, and you read nonsense

A high-impedance source — a potentiometer, a sensor behind a large resistor — charges that capacitor slowly. Convert too early and you measure a *partly charged* capacitor, not your signal.

The symptom is characteristic: readings consistently low, or changing when you touch the wiring. The fix is a longer sampling time — or a buffer amplifier.

© Munim Zabidi

Table of Contents

1 Why This Matters

2 Sampling, Quantization, and Resolution

3 The STM32 ADC

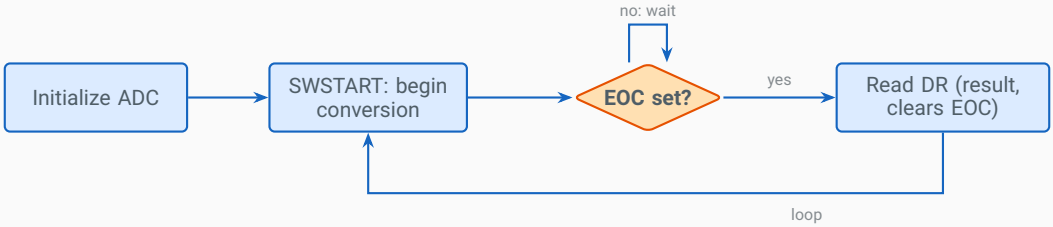
4 **Experiment: Reading a Potentiometer**

5 Interrupt-Driven Conversion

6 From Codes to Engineering Units

7 Summary

A potentiometer is a voltage divider between 3.3 V and ground. Its wiper, on `PA0` (ADC1 channel 0), gives any voltage in between. The ADC turns it into a code from 0 to 4095.



Each measurement: start a conversion, wait for `EOC`, read `DR`, repeat. Rotate the pot in the debugger: the code should run from near 0 at ground to near 4095 at 3.3 V.

© Muním Zabidi

Configuration

```
// 1. Clocks
RCC->AHB1ENR |= (1U << 0);      // GPIOA
RCC->APB2ENR |= (1U << 8);      // ADC1

// 2. PA0 as analog -- MODER bits 11, the fourth mode from Chapter 8
GPIOA->MODER |= (3U << 0);

// 3. Sampling time: 480 cycles on channel 0
ADC1->SMPR2 |= (7U << 0);

// 4. Sequence: one conversion, channel 0
ADC1->SQR1 = 0;                 // L = 0 means 1 conversion
ADC1->SQR3 = 0;                 // first in sequence is channel 0

// 5. Turn it on
ADC1->CR2 |= (1U << 0);         // ADON
```

On this course `PA0` is `ADC1` channel `0` — and note `MODER` = `11`, the analog mode that has been waiting since Chapter 8.

Reading a Value

```c
uint16_t ADC_Read(void)
{
    ADC1->CR2 |= (1U << 30);   // SWSTART
    while (!(ADC1->SR & (1U << 1)))
        ;                      // wait for EOC
    return ADC1->DR;           // reading DR
}                              // clears EOC
```

1. Start — set `SWSTART`
2. Wait — poll `EOC`
3. Read — `DR` holds the result, and reading it clears the flag

This blocks, and Chapter 9 said why that is a shame

The processor does nothing during the conversion. A real design uses the end-of-conversion interrupt, or lets DMA move the result with no CPU involvement at all.

Table of Contents

1 Why This Matters

2 Sampling, Quantization, and Resolution

3 The STM32 ADC

4 Experiment: Reading a Potentiometer

5 **Interrupt-Driven Conversion**

6 From Codes to Engineering Units

7 Summary

Interrupt-Driven Conversion

```
volatile uint32_t adc_data = 0;
volatile uint8_t  data_ready = 0;

void ADC_IRQHandler(void) {
    if (ADC1->SR & (1U << 1)) {  // EOC
        adc_data   = ADC1->DR;   // clears flag
        data_ready = 1;          // minimal work
    }
}

// in adc_init_interrupt(), as before, plus:
ADC1->CR1 |= (1U << 5);   // EOCIE
NVIC_SetPriority(ADC_IRQn, 1);
NVIC_EnableIRQ(ADC_IRQn);
```

```
while (1) {
    ADC1->CR2 |= (1U << 30); // start
    while (!data_ready) {
        // real work happens here
    }
    data_ready = 0;
    voltage = ((float)adc_data
               * 3.3f) / 4095.0f;
}
```

A 12-bit conversion at 480 cycles takes hundreds of cycles. Polling wastes them all. The `EOC` interrupt frees the CPU.

Both Programs Share a Hidden Flaw

`SWSTART` is issued from a software loop. So the gap between conversions depends on whatever the CPU was doing: a variable delay, an interrupt, one extra compiler instruction.

The sampling interval is not exactly $1/f_s$

The Nyquist rule assumes an interval of exactly $1/f_s$. A jittery sampling clock has a real f_s that is not the number you designed for.

The fix

Trigger the conversion from a hardware timer, the periodic interrupt you already built. The interval is then as exact as the timer's own clock. The exercises ask you to make this change.

Can the ADC Keep Up? Conversion Time

A timer that fires on time is useless if the conversion cannot finish before the next one is due.

$$t_{\text{conv}} = \frac{\text{sampling cycles} + \text{conversion cycles}}{f_{\text{ADC}}}$$

With the ADC at 16 MHz, a 12-bit conversion takes 12 cycles plus the sampling time. Budget: 62.5 µs per sample (16 kHz audio).

Sampling time	Cycles	t_{conv}	Share of 62.5 µs
Shortest	$3 + 12 = 15$	0.94 µs	1.5%
Longest	$480 + 12 = 492$	30.75 µs	about half

The trade-off: a high-impedance source needs a long sampling time, which spends the budget. Fix the *source impedance* instead: buffer the microphone with an op-amp. Design the signal chain backwards from the timing requirement.

Table of Contents

1 Why This Matters

2 Sampling, Quantization, and Resolution

3 The STM32 ADC

4 Experiment: Reading a Potentiometer

5 Interrupt-Driven Conversion

6 From Codes to Engineering Units

7 Summary

From Codes to Engineering Units

A thermocouple amplifier

Outputs 0–3.3 V for 0–400 °C.

```
float volts = (raw * 3.3f) / 4095.0f;
float degC  = volts * (400.0f / 3.3f);
```

Or in one step:

```
float degC = raw * (400.0f / 4095.0f);
```

Can you actually resolve 1 °C?

400 °C over 4096 steps is 0.098 °C per step — ten times better than needed.

So the ADC is not your limit. Noise is — and that is an analog design problem, not a programming one.

Note the `f` suffixes: `3.3f` is a `float`; plain `3.3` is a `double`, which the Cortex-M4's FPU cannot do in hardware.

Stage	What it produces	What can go wrong
Physical quantity	25 °C in the room	the sensor is not where you think
Sensor	250 mV	self-heating, nonlinearity, tolerance
Signal path	250 mV at the pin	source impedance, noise pickup, loading
Sampling	a held voltage	too slow for the signal; aliasing
Quantization	an integer, 0–4095	step size sets the floor on resolution
Raw code	`0x136`	reference is not exactly 3.3 V
Scaling	0.2497 V	float arithmetic, wrong constants
Engineering value	24.97 °C	calibration never applied
Reported measurement	“25 °C $\pm$ 0.5”	quoted to more digits than justified

The skill: knowing which stage you are looking at when a number is wrong.

Resolution Is Not Accuracy

Resolution

The smallest change the ADC can tell apart.

12-bit: $3.3/4096 \approx 0.8$ mV.

A property of the bit count. It never gets better than this.

Accuracy

How close the reading is to the true voltage.

Set by reference tolerance, offset, gain error, noise, source impedance, sampling time.

Can be far worse than resolution. 12-bit numbers do not guarantee 12-bit answers.

Averaging

N independent samples reduce random noise by $\sqrt{N}$. Trades conversion rate for a quieter number.

Predict, Then Measure Before you run the potentiometer experiment

Predict the code you expect at mid-travel and the voltage you expect at the pin. Then measure the pin with a meter, and read the raw code in the debugger.

If this disagrees…	the fault is in…
the meter and your prediction	the circuit
the code and the meter	the configuration: sampling time, reference, channel
the scaled result and the code	your arithmetic

Three numbers, three places to be wrong. One comparison narrows it down.

© Muním Zabidi

Application: Temperature Sensor

An LM35 gives 10 mV per °C, linearly from 0 °C:

$$T = \frac{V_{\text{IN}}}{0.01} = \frac{D \times V_{\text{REF}}}{4095 \times 0.01}$$

```c
float read_temperature_c(void) {
    uint32_t raw = adc_read();
    float volts = ((float)raw * 3.3f)
                  / 4095.0f;
    return volts / 0.01f; // 10 mV/C
}
```

Resolution check

One LSB is 806 µV, which is only 0.08 °C. Plenty.

But most of the range is wasted

The sensor spans 0.01–1.0 V of the 3.3 V range. Two thirds of the resolution is unused. An amplifier ahead of the ADC would recover it.

A light-dependent resistor in a divider with a fixed resistor R_{fixed}:

$$R_{\text{LDR}} = R_{\text{fixed}} \times \frac{V_{\text{REF}} - V_{\text{IN}}}{V_{\text{IN}}}$$

From R_{LDR} you find the illuminance through the LDR's calibration curve.

The relationship is non-linear

A lookup table in Flash, declared `const`, is often the practical way to implement it.

© Munim Zabidi

Improving a Measurement

Technique	Effect
Averaging	N samples reduce uncorrelated noise by $\sqrt{N}$. Sixteen samples cut noise by 4. It does nothing for accuracy.
Sufficient sampling time	The main cause of readings that drift or read low.
Anti-alias filtering	A simple RC low-pass ahead of the pin, with a cutoff below $f_s/2$.

© Muním Zabidi

An Analog Measurement at Five Levels

Level	Representation
Physical	1.599 V on the PA0 pad from the potentiometer wiper
Hardware	Sample-and-hold capacitor; 12-cycle SAR comparison
Register	`ADC1->DR` contains `0x7C0`
C	`voltage = raw * 3.3f / 4095.0f;`
Engineering	1.599 V, or 159.9 °C from an LM35

What the ADC gives you

A continuous quantity sensed through a chain that starts at a physical voltage and ends in engineering units. With a hardware timer driving conversions, the interval is set by the peripheral, not by the main loop.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Sampling, Quantization, and Resolution

3 The STM32 ADC

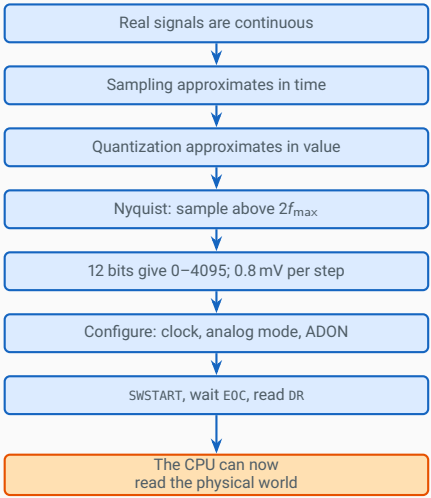
4 Experiment: Reading a Potentiometer

5 Interrupt-Driven Conversion

6 From Codes to Engineering Units

7 **Summary**

Summary



Terminology covered

- Sampling, quantization
- Nyquist, aliasing
- Resolution, step size
- Sampling time, impedance
- `EOC`, `DR`

Next: Chapter 12

UART Communication

We can read the world. Next, we report it.

© Muním Zabidi